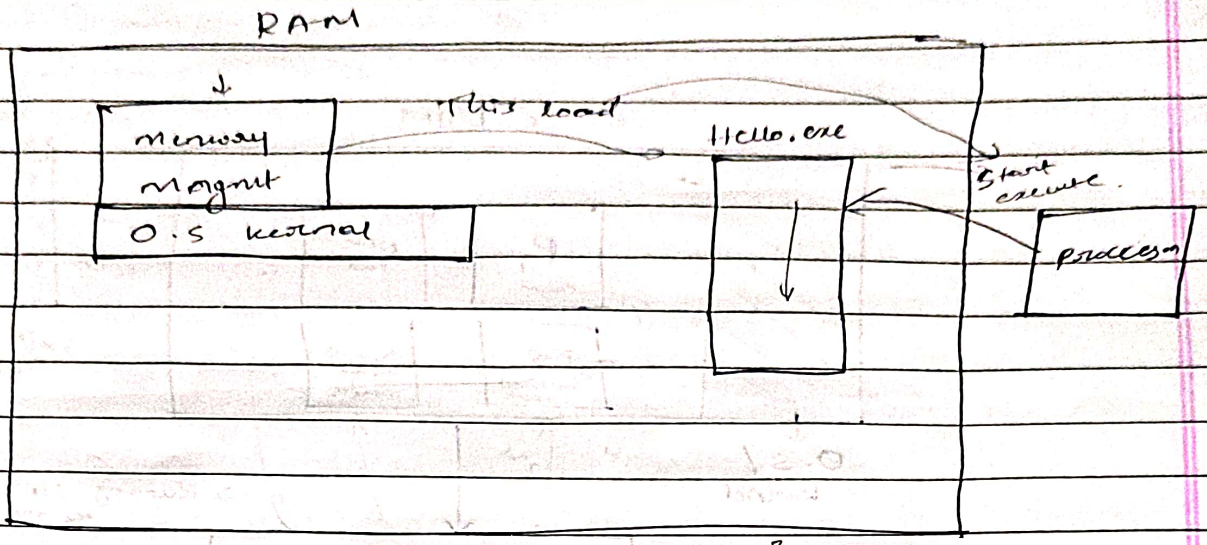
2/3/25
Day 45

13 Memory Management :-



Problem :- 1) fragmentation 2) not go beyond 4GB ram. 3)

1) Techniques :-

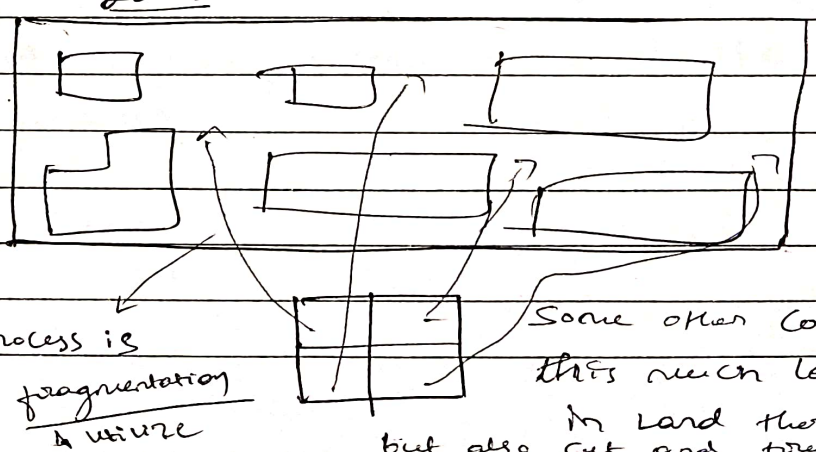
1) Segmentation -> 32 bit, old, Embedded

2) paging. -> Modern, 64 bit, Linux, Window -

This are -> the processor is start running or application start running that how will allocate the memory in RAM this are the techniques.

Why ? 1) Segmentation.

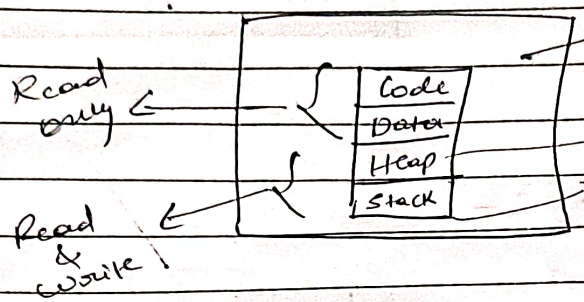
Land



SCHOOLMATE
Date
Page

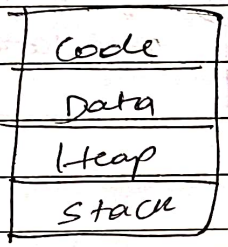
The security part :

When pogm load.



* In making O.S.

1st comes Segmentation :-



32 bit.
access address $2^{32}-1$ > 4 GB

2) Paging :- (fixed blocks) → & also equal blocks.

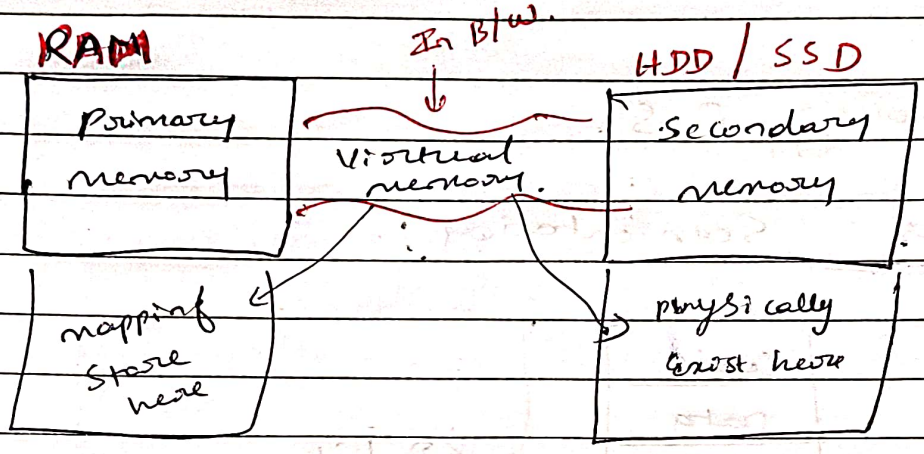
Load ⟷ RAM

* When loading pogm → they give only for the start of the pogm (Initial page)
Gives :- Data, Stack } only this much gives for start the pogm

for heap memory &
some part of the code } gives in

Virtual memory

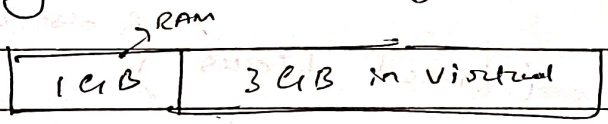
Virtual memory :-



RAM
↓
memory gives
in paging
↓
4 KB.

In RAM.

Asking 4 GB but gives.



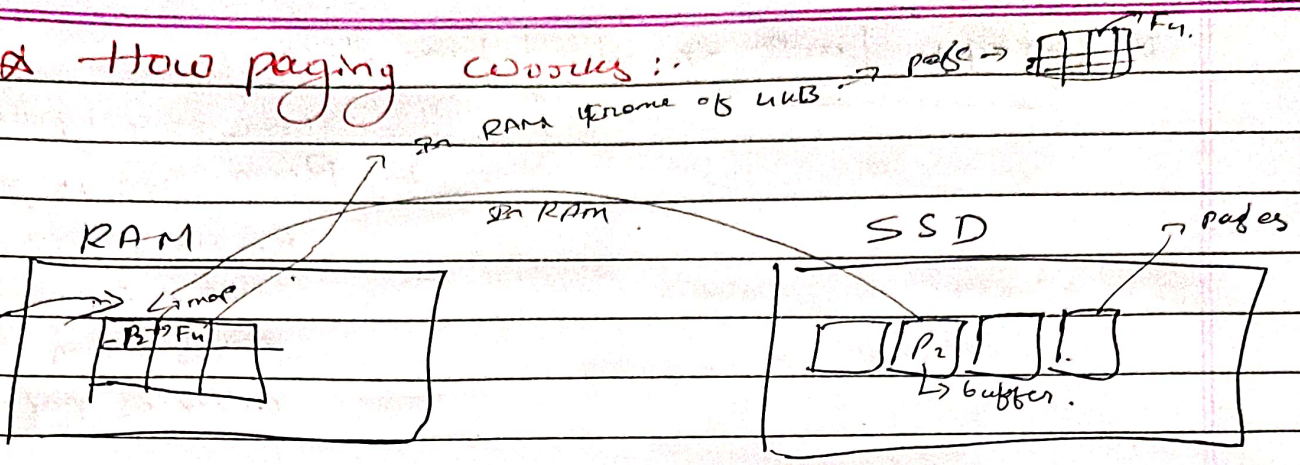
Paging :- The memory manager allocates memory to the process in the form of small chunk of pages or 4 KB.

- In RAM they called Frames
  ↓
  4 KB.

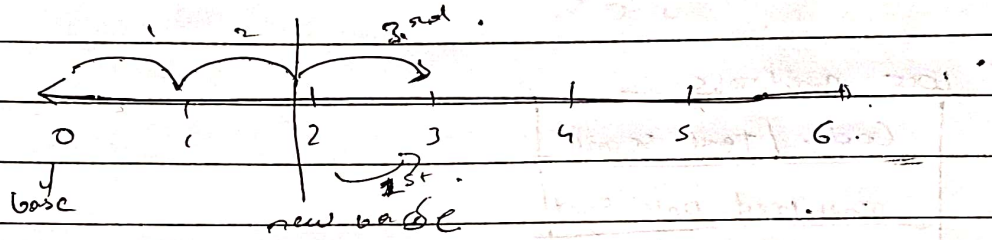
- In Virtual memory → pages

Process table
Frames → pages

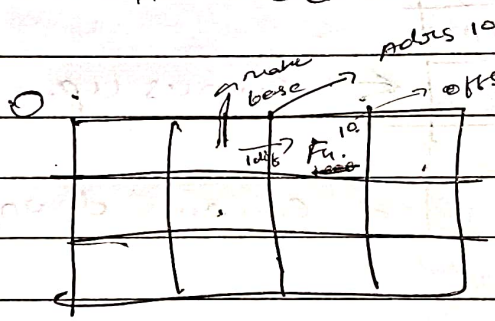
How paging works :-



offset :- In RAM.



1)



= 1000 + 10

= 1010.

1) Logical address = Page no. + offset

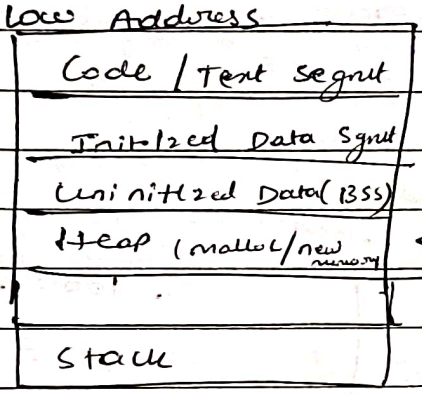
2) Page table maps pages → frames

3) Translates to physical address

RAM

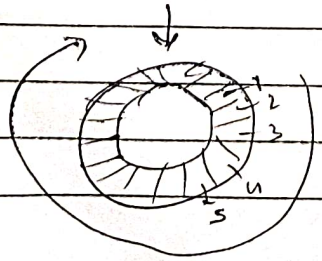
* Page fault → Load from disk
  - When there is not present in Ram then it goes then the process becomes slow. & when it not found in that called as page fault

* Memory Layout with paging :-



* Page Replacement Algorithms :- In RAM

1) FIFO → Replace oldest loaded page
2) LRU → Replace least recently accessed page
3) Optimal → place page that won't be used for longest time (ideal) (Theory not Implemented)
4) Clock → Circular queue with reference bit (2nd chance)



SCHOOLMATE
Date
Page

Memory Layout in Virtual Address Space

Symbol	Alloc at Load?	Grows?	Access
Code	✓ Yes	× No	Read + Execute
Data	✓ Yes	× No	Read + Write
Heap	× on Demand	✓ UP	Read + Write
Stack	✓ Yes	✓ Down	Read + Write

Page Table Isolation (PTI)

⇒ Why PTI?

To protect Kernal memory from user access

→ How PTI works?

- Separate page tables for user & kernel mode
- Switches on context change.

☆ Interview Questions :-

1) Paging (→ fixed block) V.S Segmentation (→ logical blocks)

2) What is a page Fault ?
   → Accessing a page not in RAM : must load from disk

3) Virtual memory ?
   L> use of disk as extension of RAM

4) FIFO Vs LRU ?
   L> . FIFO is simpler, LRU is smarter

5) what is TLB ?
   L> Hardware cache for recent page table lookups.

6) Internal Vs External Fragments ?
   L> . Internal = unused space inside block
   External = scattered small free blocks

7) Segmentation fault ?
   L> Illegal access outside segment boundaries.